- More on Sessions: https://www.quora.com/What-is-a-session-in-a-Web-Application
- More on Cookies: https://developer.mozilla.org/en-US/docs/Web/HTTP/Cookies
- Express-session Official Docs: https://github.com/expressjs/session

---

A cookie is just a **small piece of data the server tells the browser to store and send back automatically on every request.**

```
Client → POST /login → Server
Server → "Here, store this" → Set-Cookie: token=abc123
Client → GET /dashboard → Server (browser sends cookie automatically)
Server → reads cookie → "I know who this is"
```

Setting a cookie in Express

```
app.post('/login', (req, res) => {
  res.cookie('userId', '12345', {
    httpOnly: true,    // JS cannot read it — XSS protection
    secure: true,      // HTTPS only
    sameSite: 'strict',// CSRF protection
    maxAge: 24 * 60 * 60 * 1000  // 1 day in ms
  });
  res.json({ message: 'Logged in' });
});
```

Reading a cookie in Express

```
npm install cookie-parser
```

```
import cookieParser from 'cookie-parser';
app.use(cookieParser());

app.get('/dashboard', (req, res) => {
  const userId = req.cookies.userId;
  console.log(userId); // '12345'
});
```

Cookie flags — you MUST know these

Flag	What it does	Why it matters
`httpOnly`	JS can't access the cookie	Blocks XSS attacks from stealing it
`secure`	Only sent over HTTPS	Prevents interception
`sameSite: 'strict'`	Only sent to same origin	Blocks CSRF attacks
`sameSite: 'lax'`	Sent on top-level navigation	Balance of security + usability
`maxAge`	Expiry in milliseconds	Persistent vs session cookie
`domain`	Which domain receives it	Multi-subdomain auth

---

1. إيه المشكلة اللي الكوكي بتحلها؟ (The Why)

بروتوكول الـ HTTP "أقرع ونزهي" أو **Stateless**. يعني لو بعت Request دلوقتي للسيرفر وبعدين بعت واحد تاني كمان ثانية، السيرفر ميعرفش إن إنت نفس الشخص.

- **المشكلة:** اليوزر هيضطر يعمل Login في كل صفحة يدخلها!
- **الحل:** الكوكي. السيرفر بيبعت "علامة" للمتصفح، المتصفح بيشيل العلامة دي ويبعتها مع كل طلب جاي. كأنك لابس "بادج" مكتوب عليه اسمك، فالسيرفر كل ما يشوفك يعرفك.

---

2. إزاي بنبعتها؟ (Setting the Cookie)

في الـ Express، الموضوع بيتم عن طريق الـ Response Header اللي اسمه `Set-Cookie` .

JavaScript

```
res.setHeader('Set-Cookie', 'isLoggedIn=true');
```

---

3. الـ Attributes (تفاصيل "تظبيط" الكوكي)

لو بعت الكوكي كدة "سداح مداح" أي حد ممكن يسرقها أو يعدلها. عشان كدة المونجووس والـ Express بيدوك Attributes تأمن بيهم الشغل:

أ- `Expires` و `Max-Age` (العمر)

- **المشكلة:** الكوكي العادية بتموت أول ما اليوزر يقفل المتصفح (Session Cookie).
- **الحل:** بتحدد لها وقت تعيش فيه حتى لو المتصفح اتقفل.

- **الكود:** `Max-Age=3600` (تعيش ساعة بالثواني).

ب- `HttpOnly` (الحماية من الـ Hackers)

- **المشكلة:** فيه الـ XSS Attacks، الـ Hacker يقدر يكتب JavaScript كود يسرق بيه الكوكيز (`document.cookie`).
- **الحل:** الـ Attribute ده بيخلي الكوكي **مخفية** تماماً عن الـ JavaScript. المتصفح بيبعتها للسيرفر بس، ومحدش يقدر يقرأها بالكود. **(دي أهم واحدة للأمان).**

ج- `Secure` (التشفير)

- الكوكي مش هتتبعت إلا لو الموقع شغال بـ **HTTPS**. لو الموقع HTTP عادي، المتصفح هيرفض يبعتها عشان محدش يتسمع عليها في السكة (Man-in-the-middle attack).

د- `Domain` و `Path` (النطاق)

- بتحدد الكوكي دي تتبعت لمين. هل لكل الموقع؟ ولا لفولدر معين بس (مثلاً `admin/`)؟
- `Path=/admin` يعني الكوكي مش هتتبعت غير لو اليوزر داخل على صفحات الأدمن.

هـ- `SameSite` (الحماية من الـ CSRF)

- دي تريك صايعة عشان تمنع المواقع التانية إنها تبعت الكوكي بتاعتك في طلبات وهمية.
- **Strict:** الكوكي تتبعت بس لو اليوزر جوه موقعك فعلياً.

---

4. شكل الكود النهائي "المتأمن":

لما تيجي تبعت الكوكي في الـ Real World، بتبعتها كدة:

JavaScript

```
res.setHeader('Set-Cookie', 'isLoggedIn=true; Max-Age=3600; HttpOnly; Secure;
SameSite=Lax');
```

---

5. ليه مش بنستخدم الكوكيز لوحدها في الـ Authentication؟

الكوكيز ليها عيوب:

1. **مساحتها صغيرة جداً** (4KB).
2. **اليوزر يقدر يشوف الداتا اللي فيها** (لو مش مشفرة).
3. **مكشوفة للتلاعب** (اليوزر ممكن يفتح الـ DevTools ويغير `isLoggedIn=false` لـ `true`).

عشان كدة بنستخدم الـ Session: بنخزن "السر" كله على السيرفر، ونبعت لليوزر في الكوكي بس الـ **Session ID**.

من الآخر: "الكوكي هي الشنطة اللي اليوزر شايلها.. لو مأمنتهاش بـ HttpOnly و Secure، أي حد في الشارع ممكن يفتحها ويسرق اللي فيها."

```
// مثال سريع للربط بالداتابيز
const store = new MongoDBStore({
  uri: MONGODB_URI,
  collection: 'sessions'
});

app.use(session({
  secret: 'my secret',
  resave: false,
  saveUninitialized: false,
  store: store // السيشين دلوقتي متسيفة في المونجو!
}));
```

---

1. الـ `secret` (مفتاح التشفير)

ده أهم خيط في اللعبة. السيشين بتعتمد على **Session ID** بيتخزن في كوكي عند اليوزر.

- **وظيفته:** الـ `secret` ده بيستخدم عشان يعمل "Sign" للـ Cookie، يعني بيختمها بختم السيرفر.
- **ليه؟** عشان لو اليوزر حاول يعدل الـ Session ID يدوي في المتصفح، السيرفر يعرف إن الختم باظ ويرفض السيشين.
- **نصيحة "لوريستي":** في الحقيقة، الكلمة دي لازم تكون صعبة ومحطوطة في ملف `.env`.

2. الـ `resave: false` (توفير المجهود)

- **المعنى:** "هل أعيد حفظ السيشين في المخزن (Database) حتى لو مفيش أي بيانات اتغيرت؟"
- **ليه `false`؟** عشان نوفر "هلك" على الداتابيز. إحنا بنقول للمونجووس: "لو السيشين هي هي متتعبش نفسك وتسيفها تاني مع كل Request". ده بيعلي الـ Performance جداً.

3. الـ `saveUninitialized: false` (النظافة)

- **المعنى:** "هل أكريت سيشين لليوزر اللي لسه معملش Login أو مبعتش داتا تتسيف؟"
- **ليه `false`؟** عشان ملمش "زبالة" في الداتابيز. إحنا مش عايزين نفتح سيشين لكل واحد دخل يتفرج على الـ Shop ويخرج. السيشين هتبدأ "فعلياً" وتتسيف بس لما إنت تقول `req.session.user = ...` أو تغير حاجة في الـ `req.session`.

إيه اللي حصل في الـ Request دلوقتي؟ (The Workflow)

بمجرد ما السطر ده اشتغل، الـ `req` بتاعك بقى فيه Object جديد اسمه `req.session`.

1. **أول طلب:** اليوزر بيبعت Request، الميدل وير ده بيكريت له ID فريد.
2. **الرد:** السيرفر بيبعت كوكي اسمها `connect.sid` للمتصفح (مشفرة بالـ `secret`).
3. **الطلبات الجاية:** المتصفح بيبعت الـ ID ده، السيرفر بيروح يدور في "الدفاتر" بتاعته ويطلع بيانات الشخص ده ويحطها لك في `req.session`.

A cookie alone has a problem — **you're storing real data in the browser.** Anyone can open DevTools and read it. Sessions solve this.

The idea: store the real data on the **server**, give the browser only a random ID that means nothing on its own.

```
Client → POST /login
Server → creates session { userId: 123, role: 'admin' } → stored SERVER-SIDE
Server → Set-Cookie: sessionId=x9f2k (random, meaningless string)
Client → GET /dashboard → sends sessionId=x9f2k
Server → looks up x9f2k → finds { userId: 123, role: 'admin' } → responds
```

The cookie just holds a **key**. The actual data lives on the server (memory, Redis, database).

Sessions in Express with express-session

bash

```bash
npm install express-session
```

javascript

```javascript
import session from 'express-session';

app.use(session({
  secret: process.env.SESSION_SECRET,  // signs the session ID
  resave: false,
  saveUninitialized: false,
  cookie: {
    httpOnly: true,
    secure: process.env.NODE_ENV === 'production',
    maxAge: 24 * 60 * 60 * 1000
```

```
  }
}));

// Login — create session
app.post('/login', async (req, res) => {
  const user = await User.findOne({ email: req.body.email });
  // verify password...

  req.session.userId = user._id;
  req.session.role = user.role;

  res.json({ message: 'Logged in' });
});

// Protected route — read session
app.get('/dashboard', (req, res) => {
  if (!req.session.userId) {
    return res.status(401).json({ error: 'Not authenticated' });
  }
  res.json({ userId: req.session.userId });
});

// Logout — destroy session
app.post('/logout', (req, res) => {
  req.session.destroy();
  res.clearCookie('connect.sid');
  res.json({ message: 'Logged out' });
});
```

Part 3 — The Problem with Default Sessions

By default, `express-session` stores sessions **in memory** — meaning:

- Server restarts → all users logged out
- Multiple server instances → sessions don't share
- Memory leaks at scale

The fix: store sessions in Redis — and this connects directly to your Month 4 roadmap.

bash

```
npm install connect-redis ioredis
```

javascript

```javascript
import { createClient } from 'redis';
import RedisStore from 'connect-redis';

const redisClient = createClient();
await redisClient.connect();

app.use(session({
  store: new RedisStore({ client: redisClient }),
  secret: process.env.SESSION_SECRET,
  resave: false,
  saveUninitialized: false,
  cookie: { httpOnly: true, maxAge: 86400000 }
}));
```

Now sessions survive restarts and scale across multiple server instances.

الـ (The Configuration) `mongoDbStore`

أنت بتعرف الـ Store ده بره الـ `app.use` بالمنظر ده:

JavaScript

```javascript
const store = new mongoDbStore({
  uri: MONGODB_URI,        // عنوان الداتابيز بتاعتك
  collection: 'sessions',  // اسم الكولكشن اللي هيتخزن فيه السيشينز
});
```

- **التنبيه:** الـ `uri` لازم تكون هي هي اللي مستخدمها في الـ `mongoose.connect`.

إزاي المونجووس والسيشين بيتعاملوا مع بعض؟

لما يوزر يبعت Request، الـ `express-session` بتمشي في الخطوات دي:

1. بتقرأ الـ **Session ID** من الكوكي اللي جاية من المتصفح.
2. بتروح للـ `store` (اللي هو المونجو هنا) وتعمل Query: "يا مونجو، عندك سيشين بالـ ID ده؟".
3. المونجو بيرد بالبيانات، فالميدل وير يحطها لك في الـ `req.session`.

. شكل الداتا في المونجو (Inside the Database)

لو فتحت الـ MongoDB Compass وبصيت على كولكشن الـ `sessions` هتلاقي Document شكله عامل كدة:

JSON

```
{
  "_id": "vS_P9A-123456...", // الـ Session ID
  "expires": "2026-04-20T...", // تاريخ انتهاء الصلاحية
  "session": {
    "isLoggedIn": true,
    "user": { ... } // أي بيانات إنت سيفتها في req.session
  }
}
```

Sessions vs JWT — The Interview Question

This **will** come up in every interview. Here's how to answer it:

	Sessions	**JWT**
State	Server-side (stateful)	Client-side (stateless)
Storage	Server memory / Redis / DB	Browser (cookie or localStorage)
Revocation	Instant — delete from server	Hard — token valid until expiry
Scalability	Needs shared store (Redis)	Scales easily — no server state
Payload size	Tiny cookie (just an ID)	Larger — carries encoded data
Security	Server controls everything	Token theft = access until expiry
Best for	Traditional web apps, banking	APIs, microservices, mobile

The honest answer interviewers want to hear:

"Sessions are stateful — the server holds the data and the client holds a meaningless ID. This makes revocation trivial but requires shared storage like Redis at scale. JWT is stateless — all data is in the token, which makes it scalable with no shared state, but revocation is hard — you need a token blacklist which reintroduces statefulness anyway. For most web apps I'd default to sessions with Redis. For APIs consumed by mobile or third-party clients, JWT makes more sense."